Materiał dydaktyczny o ASCII i Unicode.

1) Cele i efekty

Po tym module uczeń:

- wyjaśnia różnicę między **ASCII** a **Unicode** (pojęcia: znak, punkt kodowy, kodowanie),
- rozpoznaje i używa **UTF-8/UTF-16/UTF-32**, rozumie rolę **BOM**,
- potrafi odczytać bajty znaku (hex) i odwrotnie,
- wie, skąd biorą się „krzaczki" (mojibake) i jak je naprawić.

2) Najważniejsze pojęcia (intuicyjnie)

- **Znak** – to, co widzisz (np. „A", „ą", „ 😜 ").
- **Punkt kodowy** (code point) – numer znaku w standardzie (np. U+0041 to „A").
- **Kodowanie** – sposób zapisania punktu kodowego do bajtów (np. UTF-8, UTF-16).

3) ASCII – 7 bitów, 128 znaków

ASCII (American Standard Code for Information Interchange) to kod liczbowy, który jest przyporządkowany każdemu znaku. W C++ litery, cyfry, znaki przystankowe, znaki niedrukowalne takie jak spacja czy enter przechowujemy w zmiennej typu char, mamy wtedy do dyspozycji znaki z przedziału [0; 127], oraz w rozszerzonym typie unsigned char, tu znaki zawierają się w przedziale[0; 255].

Tabela kodów ASCII

ASCII dziesiętnie	ASCII szesnastkowo	ASCII ósemkowo	znak	opis
0	0	0	Nul	zero
1	1	1	SQH	początek nagłówka
2	2	2	STX	początek tekstu
3	3	3	ETX	koniec tekstu
4	4	4	EOT	koniec transmisji
5	5	5	ENQ	wywołanie stacji
6	6	6	ACK	potwierdzenie
7	7	7	BEL	dzwonek
8	8	10	BS	backspace
9	9	11	HT	tabulacja pozioma
10	a	12	LF	przesunięcie o jeden wiersz
11	b	13	FT	tabulacja pionowa
12	c	14	FF	przesunięcie o jedną stronę
13	d	15	CR	powrót karetki - ENTER
14	e	16	SO	wyjście
15	f	17	SI	wejście
16	10	20	DLE	pominięcie znaków sterujących
17	11	21	DC1	sterowanie urządzenia 1
18	12	22	DC2	sterowanie urządzenia 2
19	13	23	DC3	sterowanie urządzenia 3
20	14	24	DC4	sterowanie urządzenia 4
21	15	25	NAC	potwierdzenie negatywne
22	16	26	SYN	synchronizacja
23	17	27	ETB	koniec bloku
24	18	30	CAN	anulowanie
25	19	31	EM	koniec nośnika

ASCII dziesiętnie	ASCII szesnastkowo	ASCII ósemkowo	znak	opis
26	1a	32	SUB	zastąpienie
27	1b	33	ESC	escape
28	1c	34	FS	poprzedza dane alfanumeryczne
29	1d	35	GS	poprzedza dane binarne
30	1e	36	RS	separator rekordów
31	1f	37	US	separator pozycji
32	20	40	SP	spacja
33	21	41	!	
34	22	42	"	
35	23	43	#	
36	24	44	$	
37	25	45	%	
38	26	46	&	
39	27	47	'	
40	28	50	(	
41	29	51	)	
42	2a	52	*	
43	2b	53	+	
44	2c	54	,	
45	2d	55	-	
46	2e	56	.	
47	2f	57	/	
48	30	60	0	
49	31	61	1	
50	32	62	2	
51	33	63	3	
52	34	64	4	

ASCII dziesiętnie	ASCII szesnastkowo	ASCII ósemkowo	znak	opis
53	35	65	5	
54	36	66	6	
55	37	67	7	
56	38	70	8	
57	39	71	9	
58	3a	72	:	
59	3b	73	;	
60	3c	74	<	
61	3d	75	=	
62	3e	76	>	
63	3f	77	?	
64	40	100	@	
65	41	101	A	
66	42	102	B	
67	43	103	C	
68	44	104	D	
69	45	105	E	
70	46	106	F	
71	47	107	G	
72	48	110	H	
73	49	111	I	
74	4a	112	J	
75	4b	113	K	
76	4c	114	L	
77	4d	115	M	
78	4e	116	N	
79	4f	117	O	

ASCII dziesiętnie	ASCII szesnastkowo	ASCII ósemkowo	znak	opis
80	50	120	P	
81	51	121	Q	
82	52	122	R	
83	53	123	S	
84	54	124	T	
85	55	125	U	
86	56	126	V	
87	57	127	W	
88	58	130	X	
89	59	131	Y	
90	5a	132	Z	
91	5b	133	[	
92	5c	134	\	
93	5d	135	]	
94	5e	136	^	
95	5f	137	_	
96	60	140	`	
97	61	141	a	
98	62	142	b	
99	63	143	c	
100	64	144	d	
101	65	145	e	
102	66	146	f	
103	67	147	g	
104	68	150	h	
105	69	151	i	
106	6a	152	j	

ASCII dziesiętnie	ASCII szesnastkowo	ASCII ósemkowo	znak	opis
107	6b	153	k	
108	6c	154	l	
109	6d	155	m	
110	6e	156	n	
111	6f	157	o	
112	70	160	p	
113	71	161	q	
114	72	162	r	
115	73	163	s	
116	74	164	t	
117	75	165	u	
118	76	166	v	
119	77	167	w	
120	78	170	x	
121	79	171	y	
122	7a	172	z	
123	7b	173	{	
124	7c	174	|	
125	7d	175	}	
126	7e	176	~	
127	7f	177		
128	80	200	€	
129	81	201	□	
130	82	202	‚	
131	83	203	□	
132	84	204	„	
133	85	205	…	

ASCII dziesiętnie	ASCII szesnastkowo	ASCII ósemkowo	znak	opis
134	86	206	†	
135	87	207	‡	
136	88	210	□	
137	89	211	‰	
138	8a	212	Š	
139	8b	213	‹	
140	8c	214	Ś	
141	8d	215	Ť	
142	8e	216	Ž	
143	8f	217	Ź	
144	90	220	□	
145	91	221	‘	
146	92	222	’	
147	93	223	“	
148	94	224	”	
149	95	225	•	
150	96	226	–	
151	97	227	—	
152	98	230	□	
153	99	231	™	
154	9a	232	š	
155	9b	233	›	
156	9c	234	ś	
157	9d	235	ť	
158	9e	236	ž	
159	9f	237	ź	
160	a0	240		

ASCII dziesiętnie	ASCII szesnastkowo	ASCII ósemkowo	znak	opis
161	a1	241	ˇ	
162	a2	242	˘	
163	a3	243	Ł	
164	a4	244	¤	
165	a5	245	Ą	
166	a6	246	¦	
167	a7	247	§	
168	a8	250	¨	
169	a9	251	©	
170	aa	252	Ş	
171	ab	253	«	
172	ac	254	¬	
173	ad	255		
174	ae	256	®	
175	af	257	Ż	
176	b0	260	°	
177	b1	261	±	
178	b2	262	˛	
179	b3	263	ł	
180	b4	264	´	
181	b5	265	µ	
182	b6	266	¶	
183	b7	267	·	
184	b8	270	¸	
185	b9	271	ą	
186	ba	272	ş	
187	bb	273	»	

ASCII dziesiętnie	ASCII szesnastkowo	ASCII ósemkowo	znak	opis
188	bc	274	Ľ	
189	bd	275	˝	
190	be	276	ľ	
191	bf	277	ż	
192	c0	300	Ŕ	
193	c1	301	Á	
194	c2	302	Â	
195	c3	303	Ă	
196	c4	304	Ä	
197	c5	305	Ĺ	
198	c6	306	Ć	
199	c7	307	Ç	
200	c8	310	Č	
201	c9	311	É	
202	ca	312	Ę	
203	cb	313	Ë	
204	cc	314	Ě	
205	cd	315	Í	
206	ce	316	Î	
207	cf	317	Ď	
208	d0	320	Đ	
209	d1	321	Ń	
210	d2	322	Ň	
211	d3	323	Ó	
212	d4	324	Ô	
213	d5	325	Ő	
214	d6	326	Ö	

ASCII dziesiętnie	ASCII szesnastkowo	ASCII ósemkowo	znak	opis
215	d7	327	×	
216	d8	330	Ř	
217	d9	331	Ů	
218	da	332	Ú	
219	db	333	Ű	
220	dc	334	Ü	
221	dd	335	Ý	
222	de	336	Ţ	
223	df	337	ß	
224	e0	340	ŕ	
225	e1	341	á	
226	e2	342	â	
227	e3	343	ă	
228	e4	344	ä	
229	e5	345	ĺ	
230	e6	346	ć	
231	e7	347	ç	
232	e8	350	č	
233	e9	351	é	
234	ea	352	ę	
235	eb	353	ë	
236	ec	354	ě	
237	ed	355	í	
238	ee	356	î	
239	ef	357	ď	
240	f0	360	đ	
241	f1	361	ń	

ASCII dziesiętnie	ASCII szesnastkowo	ASCII ósemkowo	znak	opis
242	**f2**	**362**	**ň**	
243	**f3**	**363**	**ó**	
244	**f4**	**364**	**ô**	
245	**f5**	**365**	**ő**	
246	**f6**	**366**	**ö**	
247	**f7**	**367**	**÷**	
248	**f8**	**370**	**ř**	
249	**f9**	**371**	**ů**	
250	**fa**	**372**	**ú**	
251	**fb**	**373**	**ű**	
252	**fc**	**374**	**ü**	
253	**fd**	**375**	**ý**	
254	**fe**	**376**	**ţ**	
255	**ff**	**377**	**˙**	

- Zakres: **0–127** (0x00–0x7F).
- 0–31 i 127: **sterujące** (np. 9=TAB, 10=LF, 13=CR, 27=ESC).
- 32–126: **drukowalne** (spacja, cyfry, litery A–Z, a–z, znaki interpunkcyjne).
- **Polskie litery nie mieszczą się w ASCII.**
- „Rozszerzony ASCII" (8-bit) to **lokalne strony kodowe** (np. ISO-8859-2, Windows-1250) – **nie jest standardem globalnym**.

Przydatne kody:

- spacja=32 (0x20) • „0"=48 (0x30) • „A"=65 (0x41) • „a"=97 (0x61) • „@"=64 (0x40)

4) Unicode – jedna pula znaków dla całego świata

- Zakres punktów kodowych: **U+0000…U+10FFFF** (ponad milion pozycji).
- Podział na **płaszczyzny** (BMP = U+0000…U+FFFF; dalej m.in. emoji, rzadkie pisma).
- **Unicode to repertuar znaków i numery (code points)** – **nie** surowe bajty.
- Aby zapisać do pliku, wybieramy **kodowanie Unicode** (najczęściej **UTF-8**).

Opis kodowania znaków Unicode

- **Unicode jest nazwą dla sposobu kodowania znaków w różnych językach narodowych (polski, rosyjski, turecki, chiński ...) do binarnej formy komputerowej.**

  Poprzednio wszystkie znaki były zapisywane w jednym bajcie, to znaczy, że w ten sposób było możliwe kodować ogółem 256 różnych znaków. Pierwsze 128 znaków jest przepisanych przez standard i tworzy tzw. tabelę ASCII w której jest zdefiniowane przyszeregowanie wszystkich podstawowych małych (a-z) i dużych (A-Z) liter alfabetu łacińskiego bez znaków diakrytycznych (tzn. narodowo zależne znaki), wszystkich cyfr (0,1-9), znaków specjalnych takich jak przecinek, średnik, dwukropek oraz wiele innych.
  Pozostałe 128 możliwości jest przeznaczonych do kodowania znaków specjalnych dla poszczególnych języków narodowych. Dla języków środkowoeuropejskich są to na przykład znaki á, č, ü, itd., dla języka rosyjskiego są to na przykład znaki ф, и, б, ъ, itd. Niestety, takich znaków specjalnych dla wszystkich języków na świecie jest zbyt dużo i nie można ich zakodowanie do pozostałych 128 możliwości. Z tego powodu powstały strony kodowe dla poszczególnych grup języków. Na przykład kodowanie Win-1250 dla wszystkich języków stosowanych w Europie środkowej, kodowanie Win-1251 dla wszystkich znaków Cyrilicy, itd. W ten sposób zostało rozwiązane zapisywanie tekstów dla poszczególnych języków, jednak nie było możliwe napisanie tekstu zawierającego znaki nawzajem "niekompatybilnych" języków.

  Problem kodowania wszystkich znaków we wszystkich językach został rozwiązany przez wprowadzenie kodowania Unicode (dokładniej patrz serwer konsorcjum Unicode http://www.unicode.org). To kodowanie umożliwia poprawnie zakodować wszystkie znaki na świecie. Problem został rozwiązany w ten sposób, że znaki nie są zapisywane w jednym bajcie (tylko 256 możliwości) lecz są zapisywane do 2 bajtów (tzn. 65536 możliwości). Ten sposób kodowania oznacza się identyfikatorem UTF-16. Zaletą UTF-16 jest łatwe zarządzanie wszystkimi znakami, jego mankamentem jest dwukrotna wielkość i niekompatybilność z tabelą ASCII. Taki mankament przejawia się zwłaszcza przy zapisywaniu do plików tekstowych. Z tego powodu został jeszcze zestandardyzowany następny sposób kodowania Unicode, który posiada zmienną wielkość zapisu dla poszczególnych znaków. Znaki tabeli ASCII są zapisywane w 1 bajcie, pozostałe znaki są zapisywane kolejno w 2 lub więcej bajtach (po 1. bajcie można rozpoznać, czy wystąpi kolejny bajt). Ten sposób kodowania oznacza się identyfikatorem UTF-8. Jest stosowane przeważnie dla plików tekstowych (XML, HTML). Do bezpośredniej pracy w pamięci komputera tekst taki jest transformowany do UTF-16, ponieważ praca z tekstem w tym kodowaniu jest szybsza.

  Kodowanie UTF-16 oraz UTF-8 jest stosowane w systemie PROMOTIC od wersji 7.1.0 i dlatego można wytwarzać aplikacje w różnych językach bez konieczności przełączania stron kodowych lub konieczności specjalnych wersji językowych SO Windows. UTF-16 jest stosowane podczas biegu aplikacji (teksty w panelach, w skryptach, itd.), UTF-8 jest stosowane w plikach tekstowych (np. pliki tekstów XML dla Makro wyrażenie $.text).

Link: https://symbl.cc/pl/unicode-table/#latin-1-supplement

Przykłady punktów kodowych:

- „A” → **U+0041**
- „ą” → **U+0105**
- „€” → **U+20AC**
- „ 😀 ” → **U+1F600**

5) Kodowania Unicode (skrót i przykłady)

UTF-8 (najpopularniejsze)

- Długość znaku: **1–4 bajty** (ASCII to 1 bajt – zgodność wstecz).
- Wzorce bajtów:
  1-bajt: 0xxxxxxx • 2-bajt: 110xxxxx 10xxxxxx • 3-bajt: 1110xxxx 10xxxxxx 10xxxxxx • 4-bajt: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Przykłady (hex):

- „A” (U+0041) → **41**
- „ą” (U+0105) → **C4 85**
- „€” (U+20AC) → **E2 82 AC**
- „ 😀 ” (U+1F600) → **F0 9F 98 80**

UTF-16

- Długość: **2 bajty** (BMP) albo **4 bajty** (surrogate pair dla > U+FFFF).
- **Endianness**: **LE** (little endian) / **BE** (big endian).
- Często występuje **BOM** (Byte Order Mark) na początku pliku: FF FE (LE) lub FE FF (BE).

Przykłady UTF-16LE (hex, niskobajt pierwszy):

- „A” (0x0041) → **41 00**
- „ą” (0x0105) → **05 01**
- „ 😀 ” (U+1F600) → para: D83D DE00 → **3D D8 00 DE**

UTF-32

- Zawsze **4 bajty na znak** (prosto, ale pamięciożernie).

BOM (Byte Order Mark)

- Dla UTF-8: **EF BB BF** (opcjonalny; nie zalecany w plikach kodu).
- Dla UTF-16/32: wskazuje kolejność bajtów.

6) Polska praktyka i „krzaczki”

- Dawne pliki w **Windows-1250** lub **ISO-8859-2**; przeglądane jako UTF-8 dadzą **mojibake** (np. „Zażółć gęślą jaźń” ⇒ „ZaÅ¼óÅ‚Ä‡ gÄ™Å›lÄ… jaÅºÅ„”).

- **Zasada:** Nowe projekty zapisuj w **UTF-8 (bez BOM)**; w narzędziach zawsze ustawiaj/rozpoznawaj kodowanie.

7) Ściąga (na jedną kartkę)

- ASCII: 0–127; brak polskich znaków.
- Unicode: punkty kodowe U+XXXX; różne **kodowania**: UTF-8/16/32.
- UTF-8: zmienna długość (1–4B); ASCII = 1B; emoji zwykle 4B.
- UTF-16: 2B lub 4B (surrogate); LE/BE; możliwy BOM.
- BOM: wskazuje kolejność bajtów (w UTF-8 raczej nie używać).
- „Krzaczki” = **złe kodowanie/interpretacja**.

8) Mini-laboratorium (krok po kroku)